ENSO Dashboard — Codebase Summary, Architecture and Usage

Version 1.0.0. Live at https://enso-visualization.smelnikoff.workers.dev. Snapshot of the repository after the deployment commits (branch `main`; reviewed at `f3467ec`). Companion documents: `definitions.md` (science, sourcing, classification rules), `plan.md` (feature backlog), `deployment-plan.md` (web hosting).

1. What this is

An interactive, single-page dashboard for monitoring **El Niño / La Niña** (ENSO) using four NOAA-published monthly indices and the ocean–atmosphere feedback (the *Bjerknes feedback*) that couples them.

Concept	Series	Source	Record starts
Sea-surface temperature	**ONI** (Oceanic Niño Index, Niño 3.4)	NOAA CPC `oni.ascii.txt`	1950
Atmospheric pressure	**SOI** (Southern Oscillation Index, standardized)	NOAA CPC `soi` (2nd table)	1951
Subsurface heat content / thermocline proxy	**WWV** (Warm Water Volume)	NOAA/PMEL `wwv.dat`	1980
Trade winds / Walker circulation proxy	**850 mb zonal wind index**, East / Central / West Pacific	NOAA CPC `epac850`, `cpac850`, `wpac850`	1979

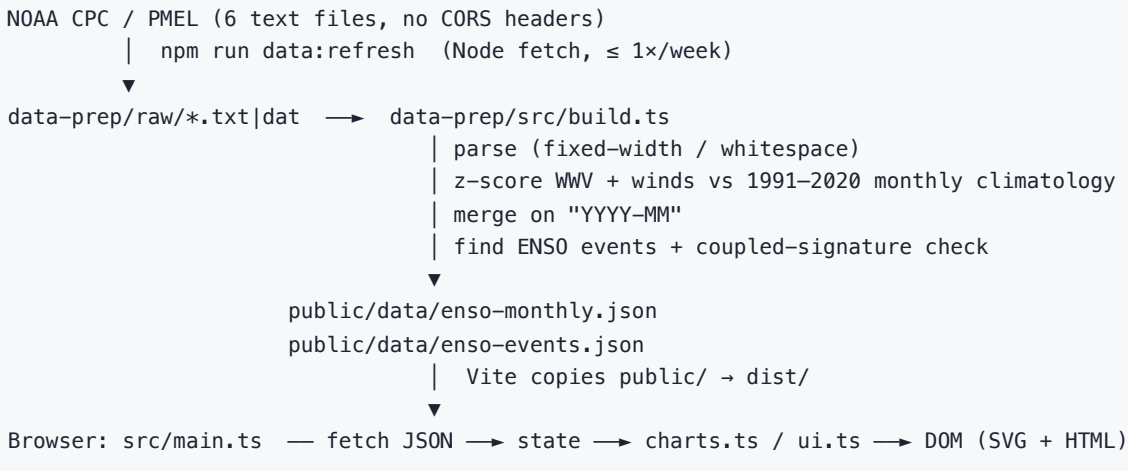
The app is **fully static**: there is no server. A Node build step downloads and processes the NOAA files into two JSON files; the browser loads those and renders everything client-side.

Stack: TypeScript (strict-ish, `erasableSyntaxOnly`), Vite 8, Observable Plot 0.6 (charts), KaTeX (math), topojson-client + world-atlas (map coastlines), `tsx` (runs the Node pipeline). No framework — the UI is plain DOM + template strings. Node 24 is in use locally.

2. Repository layout

```
ENSO_dashboard/
├── index.html                 # shell: <div id="app"> + /src/main.ts
├── package.json               # scripts (below), deps
├── tsconfig.json              # browser code (src/)
├── data-prep/                 # Node pipeline — NOT shipped to the browser
│   ├── tsconfig.json          #   separate config, targets Node
│   ├── raw/                   #   committed copies of the 6 NOAA source files
│   └── src/
│       ├── refresh.ts         #   downloads raw files (throttled to 1×/week)
│       ├── build.ts           #   parse → z-score → merge → classify → write JSON
│       ├── parseOni.ts        #   oni.ascii.txt  → monthly series
│       ├── parseWwv.ts        #   wwv.dat        → volume + anomaly series
│       ├── fixedWidthTable.ts #   column-sliced parser for soi / *850 tables
│       ├── baseline.ts        #   per-calendar-month 1991–2020 z-score
│       ├── classify.ts        #   El Niño / La Niña event detection + coupled check
│       └── types.ts
├── public/data/               # pipeline output, fetched by the browser
│   ├── enso-monthly.json      #   ~254 KB, one record per month
│   └── enso-events.json       #   ~15 KB, the event catalog
├── src/                       # the dashboard
│   ├── main.ts                #   page assembly, state, event wiring (~810 lines)
│   ├── charts.ts              #   4 Observable Plot builders
│   ├── ui.ts                  #   stat tiles, event table, data table (HTML strings)
│   ├── stats.ts               #   rolling mean, trailing regression slope, smoothRecords
│   ├── frequency.ts           #   DFT periodogram, AR1 red-noise significance, peak finding
│   ├── rangeSlider.ts         #   dependency-free dual-handle time-window slider
│   ├── format.ts              #   labels, ONI classification, date-range presets, CSS-var helper
│   ├── variableDetails.ts     #   formula + description for each coupled variable
│   ├── math.ts                #   KaTeX wrapper (renderMath)
│   ├── data.ts                #   loadData(): fetch the two JSON files
│   ├── types.ts               #   MonthlyRecord, EnsoEvent, DateRangePreset
│   └── style.css              #   design tokens (light/dark), layout, components
└── docs/                      # definitions.md, plan.md, and this summary
```

3. Architecture



Key decisions (full rationale in `definitions.md` §4)

- **Build-time data, not live fetch.** NOAA endpoints send no `Access-Control-Allow-Origin`, so browsers can't call them directly; the data also changes monthly. Hence static JSON.
- **Fixed-width parsing.** SOI and wind tables pack 6-character fields with no guaranteed separator (`…-1.4-999.9…`), so `fixedWidthTable.ts` slices by column; `-999.9` sentinels are dropped.
- **Per-calendar-month z-scores.** WWV and winds have strong seasonality; mean/std are computed per calendar month over 1991–2020 (sample std, N–1) so Junes are compared only to Junes.
- **Client-side rendering.** Observable Plot runs in the real DOM, enabling tooltips and interaction.

4. Data pipeline in detail (`data-prep/`)

Stage	File	Behaviour
Download	`refresh.ts`	Fetches 6 URLs in parallel; on failure logs a warning and keeps the committed copy (never blocks). Skips entirely if `data-prep/raw/.last-refresh` (gitignored) is < 7 days old unless `--force`.
Parse	`parseOni.ts`, `parseWwv.ts`, `fixedWidthTable.ts`	ONI season labels (DJF…NDJ) map to their centre month. SOI uses table #2 ("standardized"); winds use table #1.
Baseline	`baseline.ts`	`toMonthlyClimatologyZScore()` applied to WWV anomaly and the three wind series. SOI and ONI are used as published.
Merge	`build.ts`	Union of dates across ONI/SOI/WWV/East-wind → sorted → one `MonthlyRecord` per month with `null` for missing values.
Classify	`classify.ts`	An **event** = ≥ 5 consecutive months with ONI ≥ +0.5 (El Niño) or ≤ –0.5 (La Niña). Category from peak |ONI|: weak < 1.0 ≤ moderate < 1.5 ≤ strong < 2.0 ≤ very-strong ("Super"). **Coupled signature** `confirmed`: mean SOI < 0, WWV z > 0, East-wind z < 0 for El Niño (mirrored for La Niña); `null` if any leg lacks data (pre-1980 events).
Output	`build.ts`	Writes `enso-monthly.json` (compact) and `enso-events.json` (pretty-printed); prints record count, event count, latest ONI.

Data model (`src/types.ts`): `MonthlyRecord` = `{date "YYYY-MM", year, month, oniAnom, soiStd, wwvVolume, wwvAnom, wwvZ, windEast/Central/West Raw+Z}` (all numeric fields nullable). `EnsoEvent` = `{phase, category, startDate, endDate, peakDate, peakOni, durationMonths, coupledSignature{meanSoiStd, meanWwvZ, meanWindEastZ, confirmed}}`.

5. The dashboard (`src/`)

5.1 Page sections (top → bottom)

1. **Header** — title, subtitle, light/dark theme toggle.
2. **Smoothing bar** — 1–12 month rolling-average slider (display-only).
3. **Regional Map** — projected Pacific map with real coastlines; Niño 3.4 box tinted by ONI, WWV region tinted by heat-content z, three wind-sector boxes with arrows scaled/directed by each sector's z-score. Shows the *latest month in the current window*.
4. **ENSO Event Catalog** — "info" panel (recharge-oscillator equations in KaTeX, onset-rate formula, category legend), "view events" sortable table (period, category, peak ONI, peak month, onset rate, coupled signature), and "Jump to" preset chips. Clicking a row zooms the whole page to that event ±12 months.
5. **ONI chart** — diverging warm/cool area chart, ±0.5 and ±2.0 threshold lines, labelled super-events; "rate of change" toggle (trailing 3-month OLS slope); "info" panel; the dual-handle **time-window slider**; collapsible raw data table.
6. **Coupled Ocean–Atmosphere chart** — SOI, WWV z, East-wind z with El Niño/La Niña bands. The legend is clickable: toggling hides series and rescales the y-axis; isolating one shows its formula + definition.
7. **Frequency Analysis** — log–log DFT periodogram of the full ONI record, red-noise (AR1) significance threshold with confidence slider (60–99.9 %), labelled dominant modes, window-function buttons (rectangular / Hann / Welch / Tukey).
8. **Stat tiles** — latest ONI (or its rate), SOI, WWV z, wind z within the current window.
9. **Footer** — source links and the "all series are monthly" note.

5.2 State and rendering model

State lives in module-level variables in `main.ts` (no framework): `allRecords`, `allEvents`, `rangeStartIdx/EndIdx`, `smoothingWindow`, `visibleSeries`, `oniRateMode`, `eventSort`, `freqConfidence`, `freqWindow`, `selectedPresetKey`. Rendering is split by cost so interactions stay smooth:

Function	Re-renders	Triggered by
`renderContent()`	everything incl. tables	zoom/pan, presets, event-row click
`renderChartsAndStats()`	charts + tiles, not tables	smoothing, theme change, resize (150 ms debounce)
`renderCoupledChartOnly()`	coupled chart + legend + detail card	legend clicks
`renderOniAndStats()`	ONI chart + tiles	rate-of-change toggle
`renderEventsTableOnly()`	event table	column-header sort
`renderFrequencyChart()`	frequency chart	confidence slider, window buttons

Slider drags use `requestAnimationFrame` coalescing (`scheduleRender`). The range slider is built once and only its position is updated, so a drag's DOM is never replaced mid-gesture. The map is async (land geometry is a dynamic import) and uses a render token to discard stale results.

5.3 Computation modules

- **`stats.ts`** — `rollingMean` (trailing, null-aware), `trailingSlope` (OLS over a trailing window; null across gaps), `smoothRecords` (applies smoothing to ONI/SOI/WWV z/wind z only).
- **`frequency.ts`** — O(n²) DFT periodogram (fine at ~900 points), optional taper windows, AR1 red-noise null spectrum scaled to the periodogram's mean (Torrence & Compo 1998), threshold = red noise × –ln(1–c), dominant modes = local maxima above threshold (top 6). Whole-record only; independent of zoom/smoothing.
- **Theming** — colours are CSS custom properties; `cssVar()` reads them at chart-build time, so a theme switch just re-renders charts. Theme choice persists in `localStorage` (`enso-theme`), defaulting to the OS preference.

6. Usage

6.1 Commands

Command	Does
`npm install`	install dependencies
`npm run dev`	(pre: `data:refresh`) start Vite dev server
`npm run build`	(pre: `data:refresh`) `tsc` type-check + `vite build` → `dist/`
`npm run preview`	serve `dist/` locally
`npm run data:refresh`	download NOAA files (weekly throttle) then rebuild JSON
`npm run data:refresh:force`	same, ignoring the throttle
`npm run data:build`	rebuild JSON from existing `data-prep/raw/` only
`npm run data:typecheck`	type-check the Node pipeline

6.2 Typical workflows

- **Run locally:** `npm install && npm run dev`, open the printed URL.
- **Update data:** `npm run data:refresh:force`; reload the browser (dev mode needs no rebuild). Commit the changed `data-prep/raw/*` and `public/data/*` if you want the repo to carry the new snapshot.

- **Change a classification rule:** edit `data-prep/src/classify.ts`, run `npm run data:build`, and update the matching text in `docs/definitions.md` and `src/charts.ts` (`ONI_THRESHOLDS`) / `src/format.ts` (`classifyOniReading`) — the tier cut-offs exist in three places.
- **Verify changes:** `npx tsc --noEmit` (the project's gate), then check visually in a browser; type-checking doesn't prove charts render.

7. Deployment

Hosted on Cloudflare (Workers static assets, built from the public GitHub fork `stevexm/enso_visualization` on every push to `main`). Config lives in `wrangler.jsonc`, `.node-version` and `public/_headers`; `.github/workflows/monthly-rebuild.yml` is intended to trigger a monthly rebuild so NOAA data stays current. Details and open items: `deployment-plan.md` §14.

8. Status, known gaps and observations

Shipped (per `plan.md`): smoothing slider, custom time-window slider, KaTeX math + definitions + interactive legend, measurement-interval clarity, regional map, frequency analysis, ONI rate-of-change toggle, onset-rate column.

Open backlog (`plan.md`): onset-rate distribution view; rate-of-change for the coupled chart; finer-than-monthly data (requires gridded NetCDF sources and a new data-prep job); map layers (SST raster, wind streamlines); layout density (page is one long vertical stack, no responsive breakpoints).

Observations from the review (not bugs requested to be fixed — flagged for awareness):

1. **No automated tests.** The only gate is `tsc`; the pipeline's sanity check is the manual comparison against known strong events (`definitions.md` §2.3).
2. **Tier thresholds are duplicated** in `classify.ts`, `format.ts`, `charts.ts` and the HTML legend in `main.ts`.
3. **`rg` dependency** in `package.json` appears unused (no import in `src/` or `data-prep/`).
4. **Absolute data URLs.** `data.ts` fetches `/data/enso-*.json` from the site root, so the app must be served from a domain root (or the paths changed / Vite `base` used) — relevant to hosting.
5. **Build needs network (soft).** `prebuild` calls NOAA; if unreachable it falls back to the committed raw files, so a build never hard-fails on NOAA being down. Because `.last-refresh` is gitignored, every fresh CI build re-downloads.
6. **Stale comment:** `classify.ts` refers to `docs/data-and-definitions.md`, which is now `docs/definitions.md`; `definitions.md` §4 mentions a sibling `csv-parser/` project not in this repo.
7. **Caveat on science** (documented in `definitions.md`): NOAA's *official* monitoring has moved to RONI; this app keeps ONI for the longer record, and the strength bins are RONI's applied to ONI by convention.